Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. Deberta: Decoding-enhanced bert with disentangled attention. *arXiv preprint arXiv:2006.03654*, 2020.

K Clark. Electra: Pre-training text encoders as discriminators rather than generators. *arXiv preprint arXiv:2003.10555*, 2020.

Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

Dosovitskiy Alexey. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv: 2010.11929*, 2020.

Mahmoud Assran, Mathilde Caron, Ishan Misra, Piotr Bojanowski, Florian Bordes, Pascal Vincent, Armand Joulin, Mike Rabbat, and Nicolas Ballas. Masked siamese networks for label-efficient learning. In *European conference on computer vision*, pages 456–473. Springer, 2022.

Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers & distillation through attention. In *International conference on machine learning*, pages 10347–10357. PMLR, 2021.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

Sid Black, Leo Gao, Phil Wang, Connor Leahy, and Stella Biderman. Gpt-neo: Large scale autoregressive language modeling with mesh-tensorflow, 2021. *URL: https://doi. org/10.5281/zenodo*, 5297715, 2022.

An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. Qwen2. 5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

Myle Ott, Sergey Edunov, Alexei Baevski, Angela Fan, Sam Gross, Nathan Ng, David Grangier, and Michael Auli. fairseq: A fast, extensible toolkit for sequence modeling. In Waleed Ammar, Annie Louis, and Nasrin Mostafazadeh, editors, *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (Demonstrations)*, pages 48–53, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-4009. URL `https://aclanthology.org/N19-4009/`.

Andrey Gromov, Kushal Tirumala, Hassan Shapourian, Paolo Glorioso, and Daniel A Roberts. The unreasonable ineffectiveness of the deeper layers. *arXiv preprint arXiv:2403.17887*, 2024.

Roger A. Horn and Charles R. Johnson. *Topics in Matrix Analysis*. Cambridge University Press, 1991.

Ola Hössjer and Arvid Sjölander. Sharp lower and upper bounds for the covariance of bounded random variables. *Statistics & Probability Letters*, 182:109323, 2022.

Biao Zhang and Rico Sennrich. Root mean square layer normalization. *Advances in Neural Information Processing Systems*, 32, 2019.

Appendix

Table 2: Comparison of Post-Layer Normalization (Post-LN) and Pre-Layer Normalization (Pre-LN) Transformer Setup. MHSA = multi-head self attention, FFN = feed-forward network, LN: LayerNorm, N = number of transformer layers, x_i = input to i^{th} layer, y_i = output of ith layer, z_i & $z_{i'}$ = intermediate vectors in Post-LN model, which are inputs to LN_1 & LN_2 respectively.

Post-LN Transformer Setup	**Pre-LN Transformer Setup**
$z_i = \text{MHSA}(x_i) + x_i$ $x'_i = \text{LN}_1(z_i)$ $z'_i = \text{FFN}(x'_i) + x'_i$ $y_i = \text{LN}_2(z'_i)$	$x'_i = \text{MHSA}(\text{LN}_1(x_i)) + x_i$ $y_i = \text{FFN}(\text{LN}_2(x'_i)) + x'_i$
$y_{\text{out}} = \text{classification-head}(y_N)$ $\mathcal{L} = \text{CrossEntropyloss}(y_{\text{out}}, y_{\text{true}})$	

A Problem Setup for Gradients Analysis:

Consider a training dataset D_{train} consisting of C classes. We make an assumption that all samples in class c well represent class c. Then, we introduce a single noisy label by selecting a sample (x_1, y_c) from class c, where $c \in C$, and modify its label to a different (incorrect) label, y_{NL}, where $\text{NL}(\in C) \neq c$. The noisy sample is now represented as (x_1, y_{NL}). At the same time, we do not modify any other training samples from class c to ensure that the model has access to correctly labeled examples for effective learning as well. Next, we train a transformer model on this modified dataset until it reaches 100% training accuracy. At this stage, the model has fully memorized the noisy-labeled sample (x_1, y_{NL}) while also learning class c features from correctly labeled training samples. Now to measure the notion of memorization and learning, we compute ***memorization gradient norm*** ($\|g_x^{\text{mem}}\|_2$) and ***learning gradient norm*** ($\|g_x^{\text{learn}}\|_2$) as discussed in Sec.3.2, and compare them.

B Theorem 1: Learning Gradient Norm, $\|g_{\boldsymbol{x}}^{\text{learn}}\|_2$ is greater than or equal to Memorization Gradient Norm, $\|g_{\boldsymbol{x}}^{\text{mem}}\|_2$ across all layers.

It is formally represented as follows:

$$\left\|g_x^{\text{learn}}\right\|_2 \geq \|g_x^{\text{mem}}\|_2 \,, \qquad \text{across all layers} \tag{9}$$

Proof:

Based on the Problem Setup as discussed in Sec. A, we prove that $\left\|g_x^{\text{learn}}\right\|_2 \geq \|g_x^{\text{mem}}\|_2$ across all layers, for both Pre- and Post-LN models as follows:

B.1 *For Post-LN model:*

Firstly, we elucidate the architecture of Post-LN transformer in Tab. 2. Based on the Post-LN architecture, during backpropagation, we compute derivatives of loss w.r.t input of LN for every i^{th}-layer. Since there are two LNs in every layer, we compute and compare the learning and memorization gradients corresponding to the inputs of both LNs, i.e., $g_{z_{i'}}$ and g_{z_i} (refer to Tab. 2).

B.1.1 Backpropagating gradient analysis for LN_2 ($g_{z'_i}$):

The backpropagating gradient for $g_{z_{i'}}$ can be expressed as follows:

$$g_{z'_i} = \frac{\partial \mathcal{L}}{\partial z'_i} = \frac{\partial \mathcal{L}}{\partial y_{\text{out}}} \cdot \frac{\partial y_{\text{out}}}{\partial y_N} \cdot \prod_{j=i+1}^{N} \left(\frac{\partial y_j}{\partial z'_j} \cdot \frac{\partial z'_j}{\partial x'_j} \cdot \frac{\partial x'_j}{\partial z_j} \cdot \frac{\partial z_j}{\partial x_j} \right) \cdot \frac{\partial y_i}{\partial z'_i} \tag{10}$$

where $x_{i+1} = y_i$ because i^{th} layer's output y_i is the input of $(i+1)^{\text{th}}$ layer, x_{i+1}. To measure memorization and learning, we compute these gradients for both (x_1, y_{NL}) and (x_2, y_c), denoted as

$g^{\text{mem}}_{z_{i'}}$ and $g^{\text{learn}}_{z_{i'}}$, respectively. In both gradients, $y_{\text{out}}, y_j, z'_j, x'_j, z_j, x_j$ are only dependent on the input samples x_1 and x_2, respectively, and not on their labels, where both x_1 and x_2 genuinely represent class c. Therefore, $\frac{\partial y_{\text{out}}}{\partial y_N}, \frac{\partial y_j}{\partial z'_j}, \frac{\partial z'_j}{\partial x'_j}, \frac{\partial x'_j}{\partial z_j}, \frac{\partial z_j}{\partial x_j}, \frac{\partial y_i}{\partial z_i}$ will not be significantly different for both g^{mem}_x and g^{learn}_x, thus we regard the term, $\frac{\partial y_{\text{out}}}{\partial y_N} \cdot \prod_{j=i+1}^{N} \left(\frac{\partial y_j}{\partial z'_j} \cdot \frac{\partial z'_j}{\partial x'_j} \cdot \frac{\partial x'_j}{\partial z_j} \cdot \frac{\partial z_j}{\partial x_j} \right) \cdot \frac{\partial y_i}{\partial z'_i}$, as A_1 for both cases (as it does not vary across inputs). The only difference between the two gradients is due to $\frac{\partial \mathcal{L}}{\partial y_{\text{out}}}$, because the loss $\mathcal{L}$ is dependent on the label of both samples, i.e., y_c and y_{NL}, as follows:

$$\mathcal{L} = -\sum_{k_i=1}^{C} y_{k_i} \log(\hat{y}_{k_i}) \tag{11}$$

where $y_{k_i} = 1$ if k_i is the ground truth class, otherwise 0, and $\hat{y}_{k_i}$ is the predicted softmax probability of class k_i. As a result, $g^{\text{mem}}_{z_{i'}}$ and $g^{\text{learn}}_{z_{i'}}$ can be respectively represented as follows:

$$g^{\text{mem}}_{z_{i'}} = \frac{\partial \mathcal{L}^{\text{mem}}}{\partial y^{\text{mem}}_{\text{out}}} \cdot A_1 \quad \& \quad g^{\text{learn}}_{z_{i'}} = \frac{\partial \mathcal{L}^{\text{learn}}}{\partial y^{\text{learn}}_{\text{out}}} \cdot A_1 \tag{12}$$

Comparing learning and memorization gradients Now the problem boils down to comparing the L2-norms of $\frac{\partial \mathcal{L}^{\text{mem}}}{\partial y^{\text{mem}}_{\text{out}}}$ and $\frac{\partial \mathcal{L}^{\text{learn}}}{\partial y^{\text{learn}}_{\text{out}}}$. To do so, we need to first define $\frac{\partial \mathcal{L}}{\partial y_{\text{out}}}$ We know that $\mathcal{L}$ is the CrossEntropyLoss between the predicted softmax probability vector $\hat{y} = \text{Softmax}(y_{out})$ and the ground truth y as defined in Eq. (11). Hence, $\frac{\partial \mathcal{L}}{\partial y_{\text{out}}}$ can be written as follows:

$$\frac{\partial \mathcal{L}}{\partial y_{\text{out}}} = \hat{y} - y \tag{13}$$

Now we can understand what it really means for memorization and learning. During inference of (x_1, y_{NL}), $\hat{y_{\text{mem}}}$ will be a vector which consists of very high probability (≈ 1) for class y_{NL} while assigning extremely low probabilities (≈ 0) to remaining classes due to overfitting. Therefore, $\hat{y_{\text{mem}}} - y_{\text{NL}}$ will almost be a 0-vector, i.e., all the elements in the vector would be almost 0. This phenomenon is formally represented as follows:

$$\begin{aligned} \hat{y}_{\text{mem}} &\approx [0, \ldots, 1, \ldots, 0], \\ y_{\text{NL}} &= [0, \ldots, 1, \ldots, 0], \\ \hat{y}_{\text{mem}} - y_{\text{NL}} &\approx [0, \ldots, 0, \ldots, 0]. \end{aligned} \tag{14}$$

where the index corresponding to class y_{NL} is 1, and all other elements are (close to) 0.

Now, we can take the L2-norm of both sides in Eq. (13) for the memorizing sample. Since $\hat{y_{\text{mem}}} - y_{\text{NL}}$ is close to a 0-vector, its L2-norm ≈ 0. Therefore,

$$\left\| \frac{\partial \mathcal{L}^{\text{mem}}}{\partial y^{\text{mem}}_{\text{out}}} \right\|_2 \approx 0 \tag{15}$$

On the other hand, for (x_1, y_{c}), even though the model has learned generalizable features for class c, it has also been overfitted on noisy label samples. Hence, $\hat{y_{\text{learn}}}$ will not assign a very high probability to y_c but instead will distribute some probability mass across multiple classes. Therefore, $\hat{y_{\text{learn}}} - y_{\text{c}}$ contains non-trivial, not near-zero values. This behavior is formally presented as follows:

$$\begin{aligned} \hat{y}_{\text{learn}} &= [p_1, p_2, \ldots, p_{y_c}, \ldots, p_C], \\ y_c &= [0, \ldots, 1, \ldots, 0], \\ \hat{y}_{\text{learn}} - y_c &= [p_1, p_2, \ldots, p_{y_c} - 1, \ldots, p_C]. \end{aligned} \tag{16}$$

where p_{y_c} is not close to 1, and other probabilities p_i are non-negligible.

Now, by taking the L2-norm on both sides of Eq. (13) for the learning sample. Since $\hat{y_{\text{learn}}} - y_{\text{c}}$ contains non-trivial, non zero values, its L2-norm $\gg 0$. Therefore,